

32 | Fabric：新渲染器的演进之路

2023-1-30 蒋宏伟



你好，我是蒋宏伟。

对核心渲染流程的持续迭代和优化，是 React Native 能够广受欢迎的重要原因之一。

Fabric 是 React Native 新架构渲染器的名字。今天，这一讲我不仅要给你介绍 Fabric 渲染器的核心技术原理，更想让你通过渲染器的演变升级过程，了解该过程中 React Native 技术团队每次大升级背后的思考过程。希望这些优秀框架背后的升级思考，以及对技术极致追求的精神，能够给你带去启发。

为了便于你理解 Fabric 新渲染器是如何演变而来的，我会先和你介绍一个假想的简化版渲染器，接着再带你回顾 React Native 老架构渲染器的工作原理，最后再告诉你 Fabric 新架构渲染器是如何设计的。

简版渲染器

时至今日，在 React Native 开源之初的宏大愿景依旧打动着我：将现代 Web 技术引入移动端（Bringing modern web techniques to mobile）。

Web 开发历史悠久，沉淀了诸多优秀实践和基础设施。随着 React Web 框架的出现，将现代 Web 中积累的开发理念，以及语言、框架、规范和生态等引入移动端，统一各端基础设施，必然能够整体降低移动端学习和开发成本。这是该理念如此打动我的原因。

但难点是，如何将 Web 和移动端打通？

打通的关键是语言间的相互调用，也就是常说的语言间的通信。将 Web 技术引入移动端的关键是，打通 JavaScript 与 OC/Java 的通信，使得 JavaScript 可以调用 iOS/Android 操作系统暴露出来的 API。

我以 Hello World iOS 应用为例，给你简单介绍一下通信原理。

先看 iOS 端的原生渲染。你可以通过 Objective-C 语言调用 iOS 操作系统暴露的 API，将 Hello World 文字显示在手机屏幕上。示例代码如下：

```
// 创建一个 iOS 标签视图
UILabel *label = [[UILabel alloc] init];

// 将该标签的文案设置为 "Hello World"
label.text = @"Hello World";

// 将标签视图添加到主视图中/屏幕上
[self.view addSubview: label];
```

在 iOS 程序中，你可以通过 UILabel 控件初始化一个标签视图，并通过对 `label.text = "Hello World"` 赋值设置其文案，最后通过 `addSubview` 将其添加到主视图中，也就是将其显示到 iOS 手机屏幕上。**以上这些 API 都是操作系统提供给 iOS 应用的“渲染能力”。**

那如何将这些渲染能力暴露给 JavaScript 呢？我们在 JavaScript 引擎中提到过，开发者可以借助 JavaScript 引擎的能力，实现 JavaScript 和 C++ 函数的相互调用。而 C++ 和 Objective-C 是同一类语言，它们之间本来就可以直接调用。因此，这就实现了 JavaScript 和 Objective-C 之间的相互调用。

```
// oc
jsContext[@"setText"] = ^(string???? str) {
```

```
    label.text = str;
  };

  // javascript
  setText('Hello World')
```

例如，你可以将 `label.text = str` 装成一个 `setText` 函数，并将该函数挂在 JavaScript 引擎创建的 `JSContext` 上下文对象上。然后，在 JavaScript 代码中通过调用 `setText('Hello World')` 给标签视图赋值。

除了设置文本外，你还可以将创建视图、设置颜色/布局、添加到主视图等方法都进行封装。进一步地，任何的原生控件，都可以如上所示的一个个地封装宿主函数，并提供给 JavaScript 调用。这就是一个简版渲染器，有了它，你就实现了用 JavaScript 写原生应用的效果。

老架构渲染器

当然，上述的简版渲染器过于简单。在一个跨端渲染器中，还需要考虑组件化、声明式等开发者体验的因素，也要考虑统一 iOS/Android 布局的实现，以及如何通过多线程来提升渲染性能等多方面因素。

老渲染器最重要的职责之一，是将你在 JavaScript 侧声明的组件转换为 iOS/Android 侧的 API 命令。

如果不使用 React Native 提供的老渲染器，开发者虽然能够通过类似“简版渲染器”的方式直接调用原生 API，但这种方式显然开发效率太低。渲染器帮你把这个过程给自动化了，它可以让你用组件化的、声明式的方式描述页面，并在底层将你写的组件转化为对应原生 API。

以 Hello World 项目为例：

```
const App = () => <Text>Hello World</Text>;

AppRegistry.registerComponent(appName, () => App);
```

当你在声明一个包含 `<Text>Hello World</Text>` 的 App 组件，并将该 App 组件传给 `registerComponent` 方法之后，通过渲染器，它会将声明式的代码转换为原生命令。

以上 Hello World 应用中会包括一个用于布局的 View 视图和显示文本的视图。在 iOS 端，会生成一个 `UIView` 用于布局，并会创建 `NSAttributedString` 用于显示文本。在 Objective-C 中调用相关以上创建视图的 API 后，操作系统就会将 Hello World 文字显示在屏幕上了。

老渲染器另一个重要的职责是实现 Flex 布局。

开源第一版的 Flex 布局是直接用原生代码实现的，后来该功能独立了出来，作为了一个 C++ 第三方库 Yoga 被 React Native 引入。

当你想让 Hello World 文字居中，你可能会这么做：

```
<View style={{flex: 1, justifyContent: 'center', alignItems: 'center'}}>
  <Text>Hello World</Text>
</View>
```

给 Hello World 文字外包一层 View 组件，并在 View 组件 style 属性设置 `justifyContent`、`alignItems` 为居中。渲染器会将 style 属性设置，转化为包裹 Hello World 视图容器的 x/y 轴坐标，使其实现屏幕居中。

老渲染器还有一个职责是，尽可能地提升渲染性能。

在简版渲染器中，我们并没有单独开一个线程来执行 JavaScript 代码，因此我们的 JavaScript 代码和 UI 操作都是在主线程进行的。在主线程增加了 JavaScript 代码的执行后，整个渲染流程耗时就会增加。而且，由于 JavaScript 执行和 UI 操作是同步的，一旦 JavaScript 执行过慢，会拖慢整个渲染流程，这就大大增加了卡顿的几率。

为此，React Native 在第一版的时候，就将其设置成了双线程异步消息通信的架构。后来 React Native 团队又为 Yoga 布局引擎，并又新增了一个线程，专门用于处理布局。

整体而言，相对于单线程同步调用的架构，多线程的异步消息通信的架构，它能大幅地减少卡顿的可能性。一方面，因为，渲染任务被分解到了三个线程中，JavaScript 线程、布局线程和 UI 线程，所以 UI 线程的任务量会减少，UI 线程的渲染卡顿的几率也会减少。另一方面，采用异步通信而不是同步通信后，JavaScript 线程任务的执行不会阻塞 UI 线程。



基于此，在 2015 年 3 月，开源第一版 React Native 渲染器，在 iOS 端实现了将“Web 技术引入移动端”的目标。

Fabric 新渲染器

Fabric 新渲染器是基于老渲染器的重构升级，而重构升级过程中不变的是核心责任，是组件化/声明式、Flex布局和多线程模型。升级的是开发者体验，以及性能提升带来的用户体验。

因此，整体上看，Fabric 渲染器完成的主要任务还是将你声明的组件转换为最终原生 API 的调用。

Fabric 渲染器的转换过程主要涉及到 3 棵树：

- Element Tree
- Fiber Tree
- Shadow Tree

Element Tree

Element Tree 是在 JavaScript 侧，由 React 通过开发者书写的 JSX 创建而成的，它由若干个 Element 组成。

一般而言，根节点 `<App/>` 就是一个 Element，同时它也是一棵 Element Tree。一个 Element 就是一个普通的对象，该对象描述的是组件的实例或宿主视图的实例。

以 Hello World 应用为例：

```
const App = () => {
  return (
    <View style={{opacity: 0.99, flex:1, justifyContent: 'center', alignItems: 'center'}}>
      <Text style={{opacity: 0.88}}>Hello World</Text>
    </View>
  );
};

// Element Tree
<App/>
```

整个应用的根节点是 `<App/>`，`<App/>` 子节点是 `<View/>`，`<View/>` 的子节点是 `<Text/>`，它们共同构成了一棵 Element Tree。

一棵 Element Tree 的每个节点都是一个 Element。React Element 有两种类型，一种是通过函数或类自定义的合成组件生成的，另一种是宿主组件生成的。其中，宿主组件指的框架通过 JavaScript 引擎暴露给 JavaScript 的原生组件。

`<App/>` 根节点是自定义函数创建的，所以它是合成组件生成的节点。`<App/>` 根节点打印出来如下：

createFiberFromElement element:

▼ Object

```
Symbol(react.element)
_owner: null
_self: undefined
_source: undefined
_store: {validated: true}
key: null
props: {concurrentRoot: true, rootTag: 1}
ref: null
type: function()
  arguments: null
  caller: null
  length: 0
  name: "App"
  prototype: App {}
  Function Prototype
```

► Object Prototype

它是一个由 type、props、concurrentRoot 等属性组成的对象。其中，type 属性是一个 function () 函数，函数名 name 是 App。

而 <Text/> 节点是由框架暴露组件生成的节点，<Text/> 元素打印出来如下：

▼ createFiberFromElement element:

▼ Object

```
Symbol(react.element)
_owner: FiberNode {tag: 11, key: null, elementType: Object, type: Object, stateNode: null, ...}
_self: undefined
_source: undefined
_store: {validated: true}
key: null
props: Object
  accessibilityState: undefined
  accessible: true
  allowFontScaling: true
  children: "Hello World"
  disabled: undefined
  ellipsizeMode: "tail"
  isHighlighted: false
  numberOfLines: undefined
  selectionColor: null
  style: {opacity: 0.88}
  Object Prototype
ref: null
type: "RCTText"
  Object Prototype
```

从上图可知，一个 Element 也是一个普通的对象。该对象的 type 属性值为字符串 RCTText，style 属性值由设置透明属性 opacity: 0.99 和设置居中布局的属性组成，子节点 children 属性值为 Hello World。

从 Hello World 应用中的 <App> <Text> 节点的构成，我们可以看出，一个 Element 常见的属性包括 type、props、currentRoot、style、children 等属性。

type: type 代表该 Element 的类型。如果 type 的值是 RCTText、RCTView 之类的字符串，那么该 Element 对应着一个宿主视图。如果 type 的值是函数或类，那么该 Element 是由合成组件生成的，并且没有对应的宿主视图。

props: Element 初始化传入的属性，其中又包括当前根节点 currentRoot、样式 style、子节点 children，或者例如 Text 组件的 ellipsizeMode 文本省略属性等等。

在 React 层，Element Tree 会被映射为 Fiber Tree。

Fiber Tree

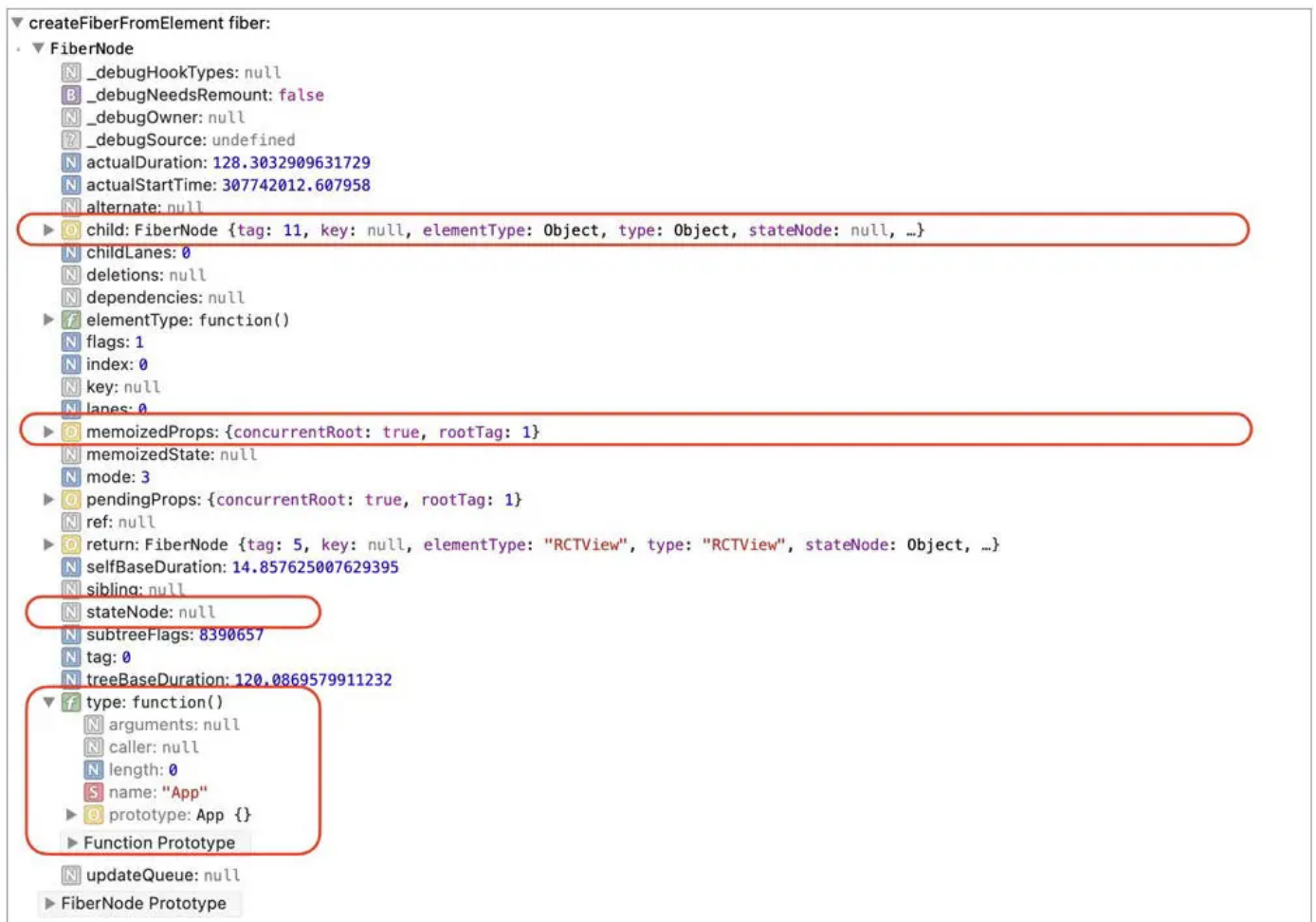
Fiber Tree 是由若干个 Fiber 节点组成的，如果某个 Fiber 节点是通过用于描述宿主视图的 Element 生成的，那么该 Fiber 会对应一个同样的宿主视图。

Fiber 是 React 16 之后引入的新能力，它使得 React 每次可渲染的颗粒度更小了，由 React 16 之前的一次 render 所有节点，变为了一次 render 时可分批次对节点进行操作。因此，从渲染角度，我们还可以将 Fiber 节点看做每次 render 的最小的渲染单位，让它能 Fabric 渲染器更快更智能。

在 React 内部，Fiber 节点是由 createFiberFromElement 函数创建的。从名字上可以看出，Fiber 节点是由 Element 节点生成的。进一步地，Fiber Tree 也可以看做 Element Tree 的映射。

同样，Fiber 节点也分两种，一种是由合成组件生成的 Element 所映射的 Fiber 节点，它没有对应的宿主组件的实例；一种是由宿主组件生成 Element 所映射的 Fiber 节点，它拥有对应的宿主组件实例。

还是以 Hello World 应用为例。你可以将 App 组件所创建的 Fiber 节点，打印出来如下：

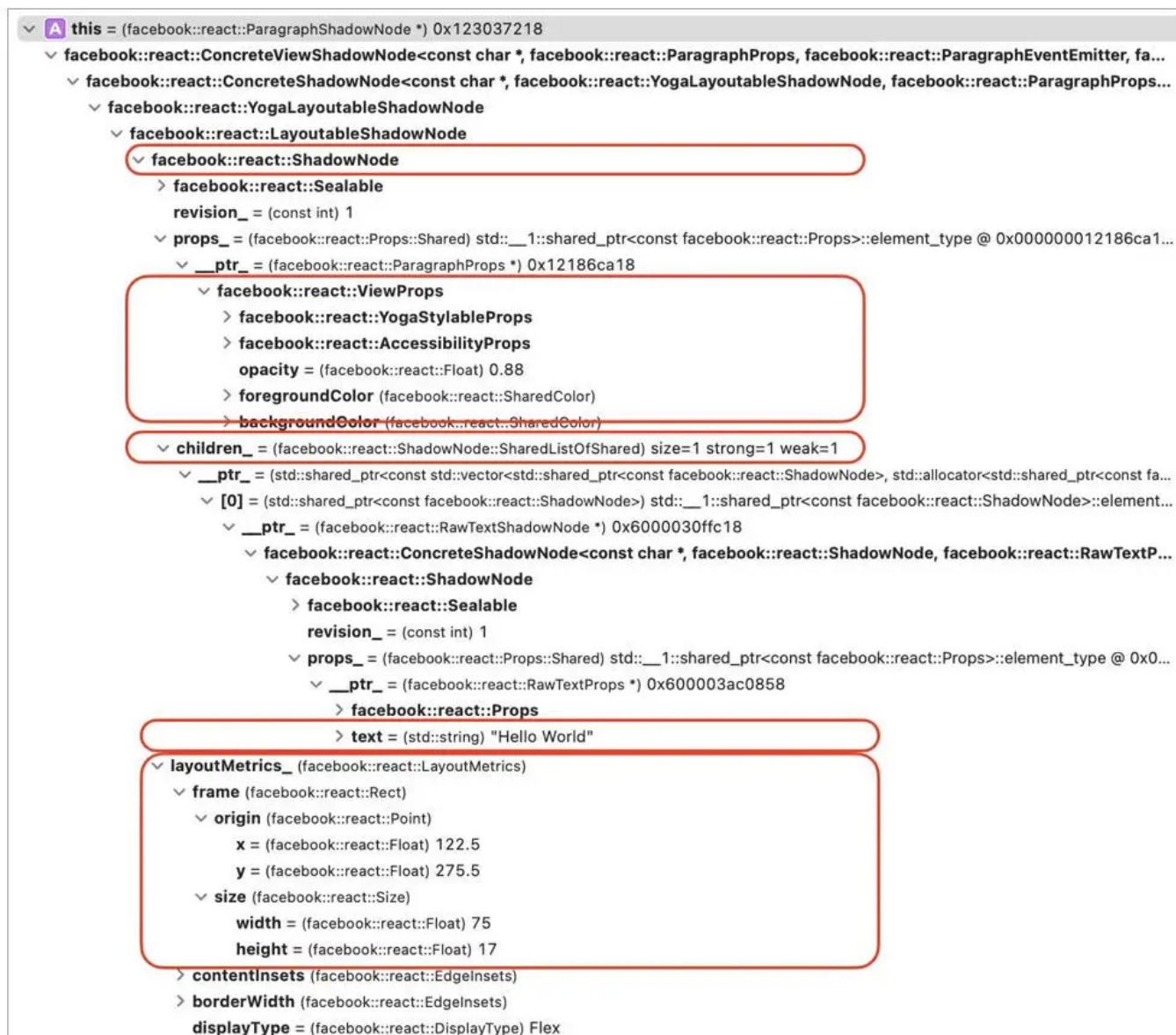


可以看到，App Fiber 节点是一个对象。该对象也有类型 type、子节点 child、属性 *props 等对象属性，这些对象属性在 Element 上也有。例如，App 组件所创建的 Element、Fiber 的 type 都是一个名为 App 的函数。当然，Fiber 节点上的属性是比 Element 多，比如 Fiber 节点拥有兄弟节点 sibling、父节点 return、状态节点 stateNode 等属性。

状态节点 stateNode 是一个较为特殊的属性，它关联了渲染器在 C++ 层生成的 Shadow 节点。App Fiber 节点的 stateNode 为 null，代表的就是合成组件所对应的 Fiber 节点是没有关联 Shadow 节点的，也就没有对应的宿主视图了。

同时，我也将 Hello World 应用中通过 Text 组件所创建的 Fiber 节点，打印了出来，如下：

在 Xcode 中，它对应的 Shadow 节点打印出来如下：

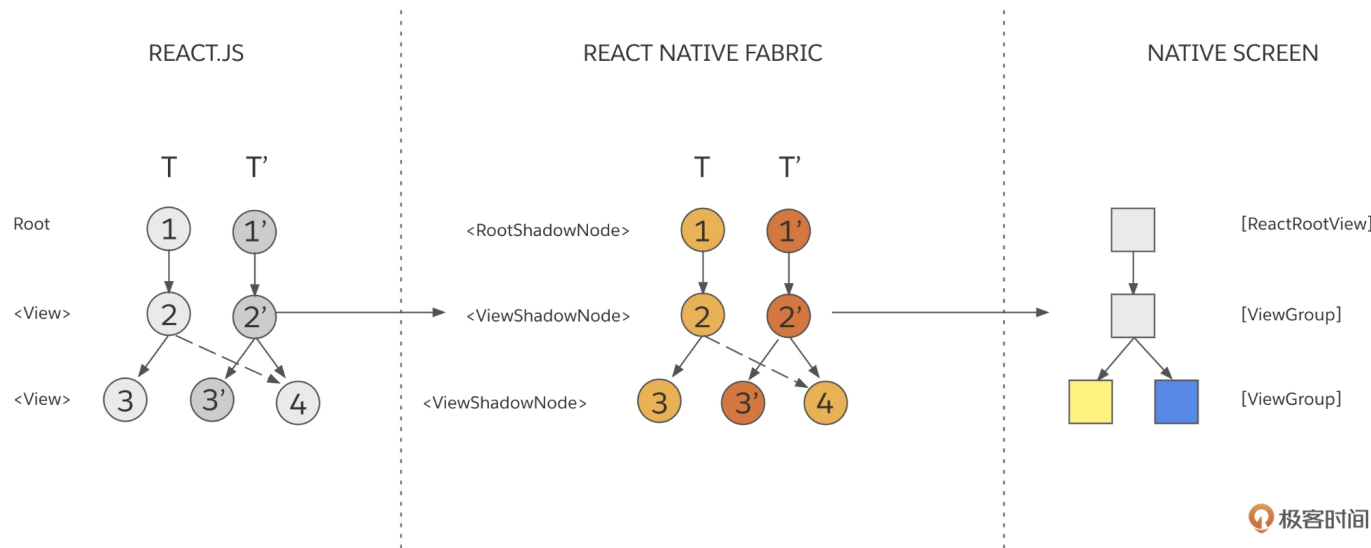


<Text> 元素对应的 Shadow 节点是个原生对象，该对象上挂载了属性 props、子节点 children、布局 layoutMetrics 等对象属性。

其中，Shadow 的 props 的透明度 opacity: 0.88 来自于 JSX 中的 style={{opacity: 0.88}} 的设置；子节点 children 的 text="Hello world" 来自于 JSX 标签括起来的内容 Hello World。而 x/y 轴坐标以及 width/height 视图大小是根据其自身 style 布局属性，以及父节点和其他节点 style 布局属性计算出来的。

也就是说，Shadow Tree 不仅继承了由 JSX 所创建 Element Tree 的相关属性、父子节点关系，还新增了该视图如何在屏幕上进行布局的具体值。

最后，在 Fabric 渲染器的 C++ 层，通过 Diff 算法对比更新前后的两棵 Shadow Tree，计算出更新视图的操作指令（这些操作指令类似于简版渲染器中提到的 `setText('Hello World')`），完成最终的渲染。这就是，Fabric 渲染器实现将 JSX 渲染成原生视图的整体流程。



更多的细节，可以参考我翻译的官方文档 [《Fabric 渲染流水线》](#)。

总结

从老渲染到 Fabric 新渲染器，虽然代码几乎完全重写，但它坚守的初衷始终没有变过，而且 Fabric 新渲染器更好地完成了“将 Web 技术带到移动端”的使命。

Fabric 渲染器使用的是 React 16 以上版本，通过 Fiber 的能力，降低了大批量渲染卡顿的可能性。通信方式由异步消息升级为了同步调用，这也减少了通讯的性能损耗。多线程模型支持了 6 种不同优先级的渲染模式，由此可针对不同场景采用不同的渲染方案，这也进一步减少了卡顿的几率。

因此，我们可以说 Fabric 新渲染器就是对老渲染器在开发体验和渲染性能上的演化升级。

思考题

社区一些开源库更新一段时间后就逐渐停止了维护，而 React Native 持续迭代和优化了 8 年之久，你认为是什么因素导致了它有如此强的生命力？

欢迎在留言区分享你的看法、交流学习心得或者提出问题，如果觉得有收获，也期待你把今天的内容分享给更多的朋友！

上一篇 31 | 多线程：RN底层是如何保障交互体验的？

精选留言 0